BUT2 INFORMATIQUE · RESSOURCE R3.05

Programmation système

CM 06 — L'ordonnanceur

```
bash — R3.05
$ nice -n 10 ./calcul &
[1] 5120
$ ps -o pid,ni,pri,cmd -p 5120
  PID  NI PRI CMD
 5120  10  29 ./calcul
```

Christophe Cruz · christophe.cruz@ube.fr · 2026-2027

IUT Dijon-Auxerre · Université Bourgogne Europe

PLAN

Au programme

01

Pourquoi ordonnancer ?

La multiprogrammation, le comportement des processus, les moments où l'on choisit.

02

Critères et types

Temps d'attente, temps de séjour, temps de réponse ; préemptif ou non.

03

Les algorithmes

FIFO, SJF, SRTN, tourniquet, priorités, files multiniveaux, avec diagrammes de Gantt.

04

En pratique sous Linux

CFS puis EEVDF, les classes temps réel, nice et renice.

Prérequis : CM 03a (états d'un processus et transitions, mode noyau).

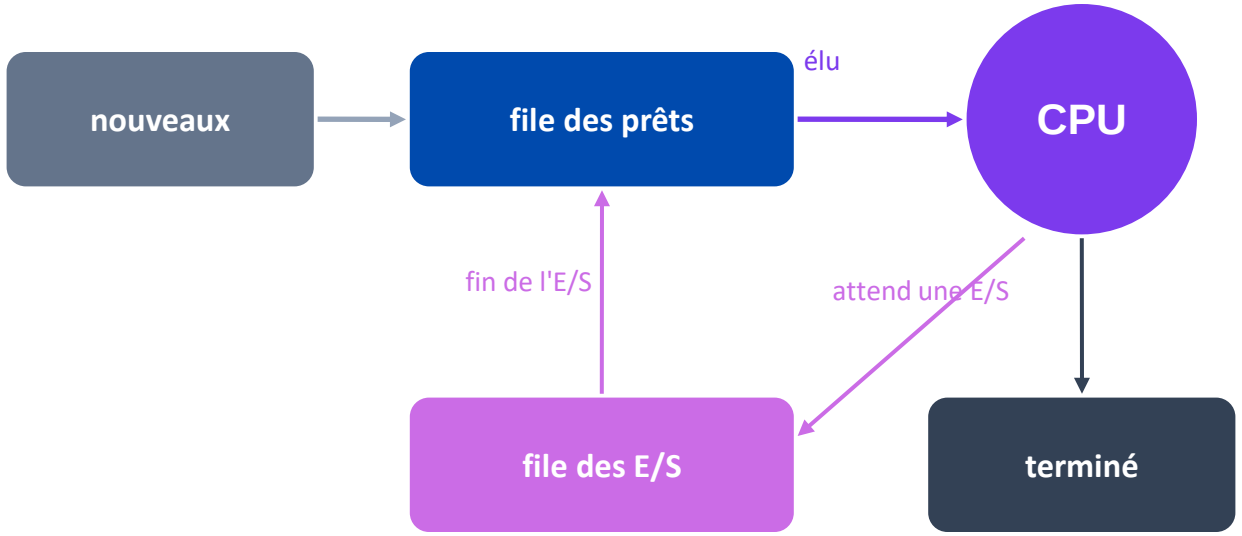
01

Pourquoi ordonnancer ?

Plusieurs processus prêts, un seul processeur

Qui va utiliser le processeur ?

- La multiprogrammation met les processus en concurrence pour le processeur
- Sur un cœur, un seul processus s'exécute à la fois ; les autres attendent dans la file des prêts
- L'ordonnanceur (scheduler) et son algorithme décident lequel s'exécute



Lien avec le CM 03a : c'est la transition (2) « élu par l'ordonnanceur » du diagramme des états. Le passage d'un processus à l'autre s'appelle un changement de contexte.

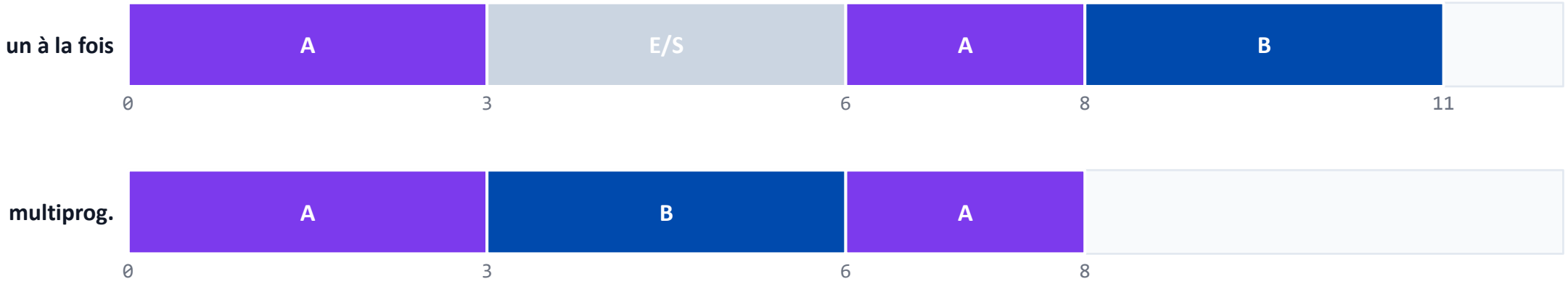
Hier et aujourd'hui

Hier : un travail à la fois

Un programme se termine, on lance le suivant (IBM System/360, années 1960). Pendant une E/S, le processeur attend sans rien faire.

Aujourd'hui : la multiprogrammation

Le processeur est bien plus rapide que l'utilisateur et que les disques : pendant qu'un processus attend, un autre s'exécute.



B se termine à 6 au lieu de 11 : le temps d'E/S de A est utilisé.

Deux comportements de processus

Tributaire du processeur
calcul, compression, compilation



rafales longues de calcul, rares E/S

Tributaire des E/S
éditeur, navigateur, serveur



rafales courtes, beaucoup d'attente d'E/S

Plus les processeurs sont rapides, plus les processus deviennent tributaires des E/S : le calcul est vite fini, il reste l'attente. Bien les ordonnancer garde les disques et le réseau occupés.

Exemple

Vous envoyez un courriel et fermez l'application. Si la fermeture prend 2 s, l'ordinateur semble lent. Si la fenêtre se ferme tout de suite et l'envoi part 2 s plus tard, vous êtes satisfait. Bien choisir, c'est cela.

Quand l'ordonnanceur intervient-il ?

1. **Création :** un fils vient d'être créé (fork) : exécuter le père ou le fils ?
2. **Terminaison :** le processus élu se termine : il faut en choisir un autre
3. **Blocage :** le processus attend une E/S, un sémaphore, un fils…
4. **Fin d'E/S :** une interruption réveille un processus : doit-il passer devant ?
5. **Horloge :** le quantum de temps est écoulé (ordonnanceur préemptif)

Dans tous les cas, une seule question :

à qui donne-t-on le processeur ?

02

Critères et types

Comment juger un algorithme d'ordonnancement

Les critères d'ordonnancement

Critère	Définition	On veut
Utilisation du CPU	% du temps où le processeur travaille	maximiser
Débit	nombre de processus terminés par unité de temps	maximiser
Temps de séjour	fin − arrivée	minimiser
Temps d'attente	temps passé dans la file des prêts = séjour − exécution	minimiser
Temps de réponse	arrivée → première réaction visible	minimiser
Équité	chaque processus obtient sa juste part	garantir

Attention au vocabulaire
Temps de séjour (turnaround) : toute la vie du processus.
Temps d'attente : seulement le temps passé à attendre le processeur.
L'ancien cours les confondait (voir FIFO).

Ces critères se contredisent : impossible de tous les optimiser à la fois.

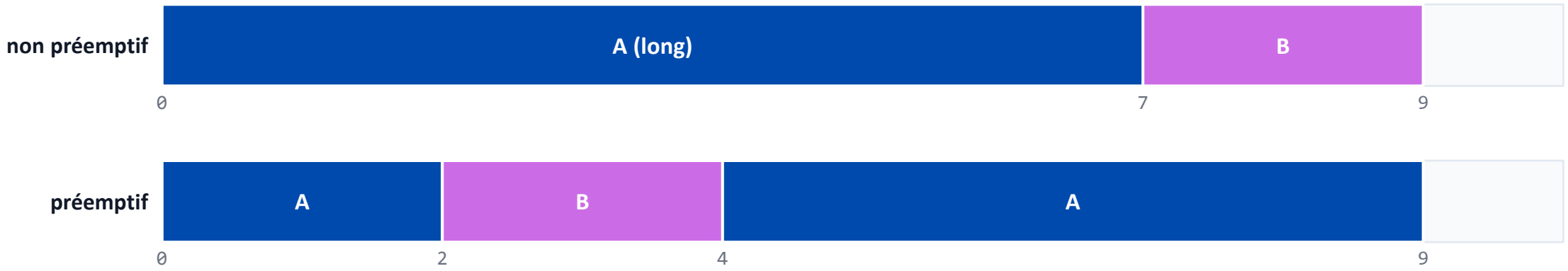
Préemptif ou non préemptif

Non préemptif

Le processus élu garde le processeur jusqu'à ce qu'il se bloque, se termine ou le rende lui-même.

Préemptif

Le processus élu s'exécute au plus pendant un délai (quantum). Si le délai expire, il est suspendu et un autre est choisi.



B est un processus interactif court : avec la préemption, il répond à t = 4 au lieu de 9.

Des objectifs selon le système

Tous les systèmes

Équité : un temps juste pour chacun, dans le respect de la politique du système.

Équilibre : occuper toutes les parties du système.

Traitement par lots

Débit et temps de séjour. Ex. : calculs scientifiques, sauvegardes nocturnes.

Systèmes interactifs

Temps de réponse : réagir vite aux requêtes, répondre aux attentes des utilisateurs.

Temps réel

Respecter les échéances. Prévisibilité : pas de saccade dans un flux vidéo, pas de retard dans un freinage ABS.

03

Les algorithmes

Des diagrammes de Gantt pour comparer

Six algorithmes classiques

FIFO

premier arrivé, premier servi

non préemptif

SJF

le plus court d'abord

non préemptif

SRTN

le plus court temps restant

préemptif

Tourniquet

Round Robin : chacun son quantum

préemptif

Priorités

la plus haute priorité d'abord

préemptif

Files multiniveaux

plusieurs files, plusieurs politiques

préemptif

FIFO : premier arrivé, premier servi

Processus	Durée
P1	24 ms
P2	3 ms
P3	3 ms

Tous arrivent à t = 0

Ordre P1, P2, P3



attente : (0 + 24 + 27) / 3 = **17 ms** séjour : (24 + 27 + 30) / 3 = 27 ms

Ordre P2, P3, P1



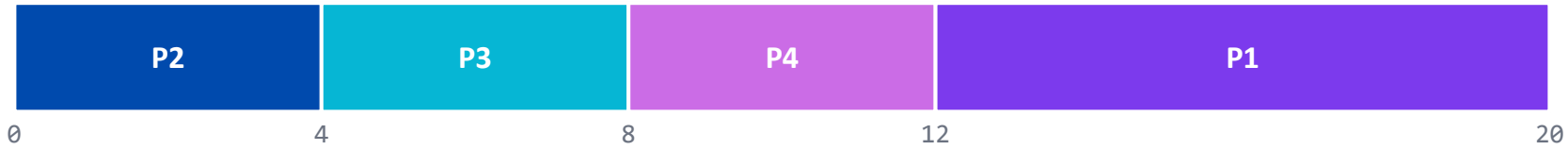
attente : (0 + 3 + 6) / 3 = **3 ms** séjour : (3 + 6 + 30) / 3 = 13 ms

Effet convoi : les petits processus attendent derrière un gros. FIFO est simple et équitable, mais très dépendant de l'ordre d'arrivée. (L'ancien cours appelait « temps d'attente » les valeurs 27 et 13, qui sont des temps de séjour.)

SJF : le plus court d'abord

Processus	Durée
P1	8 ms
P2	4 ms
P3	4 ms
P4	4 ms

Shortest Job First



attente : (0 + 4 + 8 + 12) / 4 = **6 ms** séjour : (4 + 8 + 12 + 20) / 4 = 11 ms

Optimal

SJF donne le plus petit temps d'attente moyen… si tous les processus sont présents dès le départ.

Il faut connaître la durée

Le système ne la connaît pas : il l'estime, par exemple par une moyenne des rafales précédentes.

Famine

Si des processus courts arrivent sans cesse, un processus long risque de ne jamais s'exécuter.

SRTN : le plus court temps restant

Processus	Arrivée	Durée
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Shortest Remaining Time Next (préemptif)



`t = 2 : P2 (4) < reste de P1 (5) → préemption`

`t = 4 : P3 (1) < reste de P2 (2) → préemption`

`t = 5 : restes P2 = 2, P4 = 4, P1 = 5 → P2`

attente : P1 = 9, P2 = 1, P3 = 0, P4 = 2 → moyenne **3 ms**

À chaque arrivée, on compare la durée du nouveau venu au temps restant du processus élu. Il favorise les travaux courts, mais il faut toujours connaître les durées à l'avance, et les processus longs peuvent souffrir de famine.

Le tourniquet (Round Robin)

- Les processus prêts tournent dans une file
- Chacun s'exécute pendant un quantum de temps
- Fin du quantum : s'il n'a pas fini, il retourne en fin de file
- S'il se termine ou se bloque avant, le suivant passe aussitôt

P1 = 24, P2 = 3, P3 = 3, quantum = 4



attente : P1 = 6, P2 = 4, P3 = 7 → **5,67 ms**

FIFO dans le même ordre : 17 ms. Les processus courts n'attendent plus derrière P1.

Équitable et réactif

Aucun processus n'attend plus de (n − 1) × quantum avant de s'exécuter : c'est la base des systèmes interactifs. En revanche, le temps de séjour moyen est souvent moins bon qu'avec SJF.

Comment choisir le quantum ?

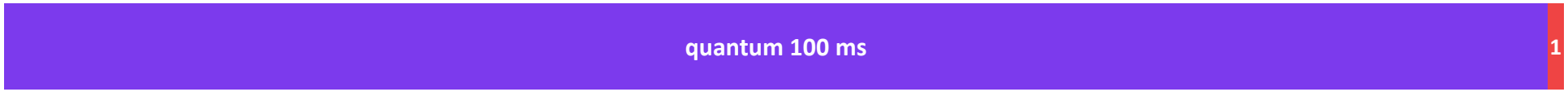
Chaque changement de contexte coûte du temps (sauvegarder les registres, changer de table des pages…). Supposons 1 ms par changement.

q = 4 ms



1 / (4 + 1) = 20 % du temps perdu en commutations

q = 100 ms

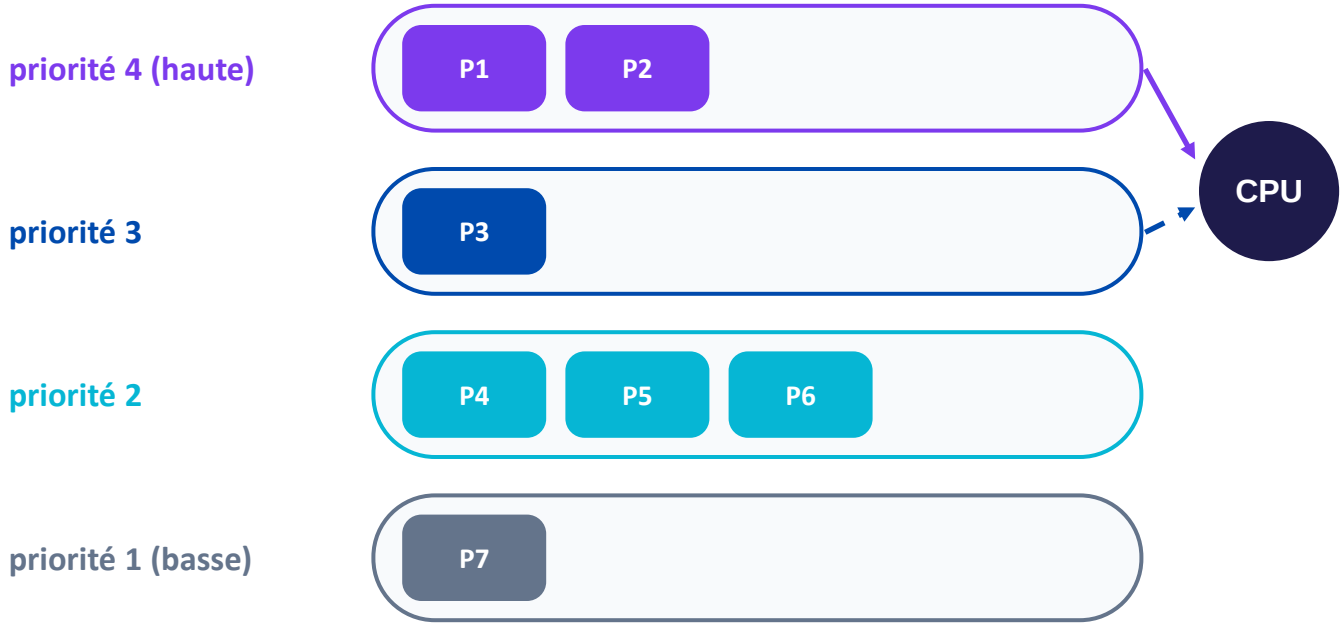


1 / 101 ≈ 1 % perdu… mais avec 10 processus prêts, le dernier attend près d'une seconde

Compromis : un quantum trop court gaspille le processeur, un quantum trop long rend le système peu réactif. On choisit en général entre 10 et 50 ms ; Linux ajuste dynamiquement des tranches de quelques millisecondes.

L'ordonnancement par priorités

- Une file par niveau de priorité
- Chaque file est gérée en tourniquet
- On sert d'abord la file de priorité maximale
- Quand elle est vide, on passe à la file inférieure

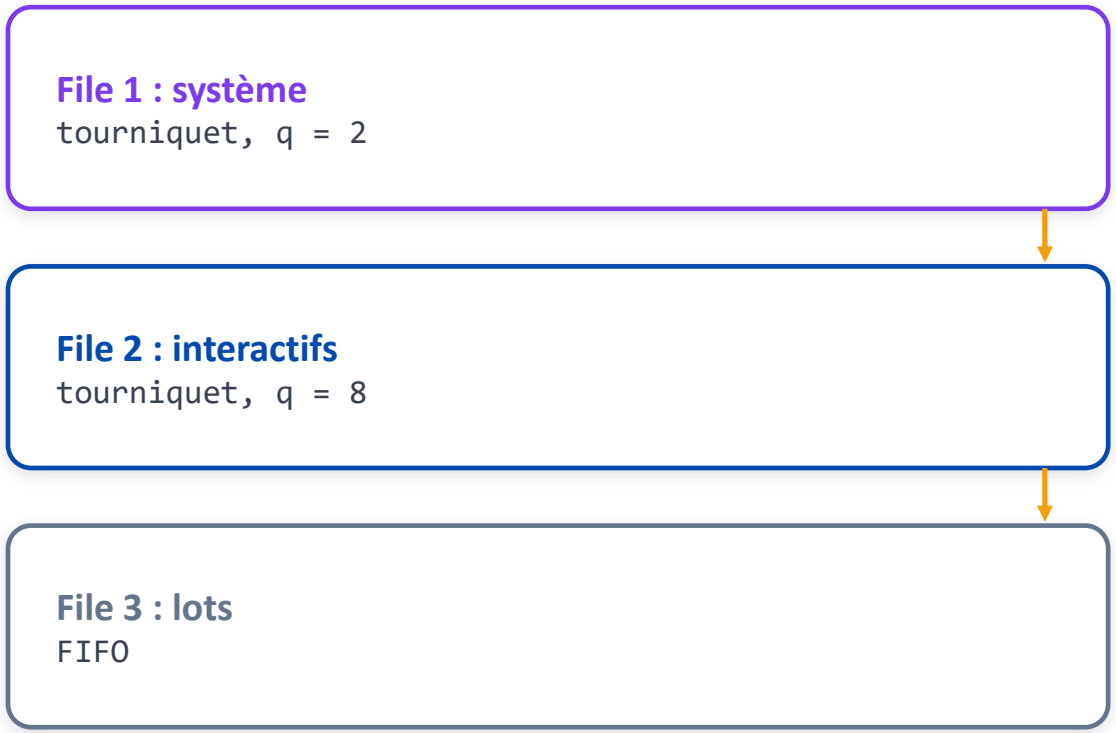


puis la file 3, quand la 4 est vide

Risque de famine : P7 peut ne jamais passer. Solution : le vieillissement (aging), qui augmente peu à peu la priorité des processus qui attendent longtemps.

Les files multiniveaux

- Plusieurs files, chacune avec son propre algorithme
- Chaque file a un but : premier plan (interactif) en tourniquet, arrière-plan (lots) en FIFO…
- Chaque processus est affecté à une file à sa création
- Variante avec rétroaction : un processus qui consomme tout son quantum descend d'un niveau, un processus interactif remonte



descend si le quantum est épuisé

Récapitulatif

Algorithme	Préemptif	Durée connue ?	Avantage	Inconvénient
FIFO	non	non	simple, équitable	effet convoi
SJF	non	oui	attente moyenne minimale	famine des longs
SRTN	oui	oui	favorise les courts	famine, estimation
Tourniquet	oui	non	réactif, équitable	choix du quantum
Priorités	oui	non	traite l'urgent d'abord	famine (→ vieillissement)
Multiniveaux	oui	non	adapté à chaque type	complexe à régler

Exercice : calculez le temps d'attente moyen

Processus	Durée
A	5
B	2
C	8
D	3

Arrivée de tous à t = 0, dans l'ordre A, B, C, D

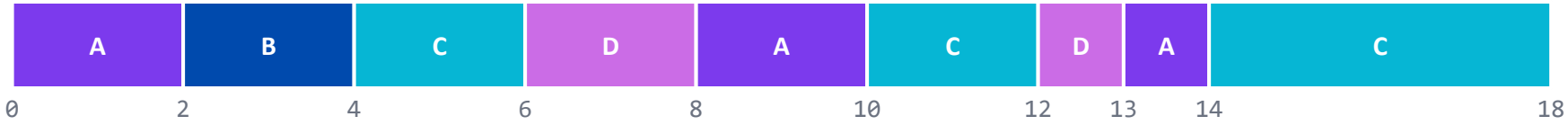
FIFO — attente : (0 + 5 + 7 + 15) / 4 = 6,75



SJF — attente : (0 + 2 + 5 + 10) / 4 = 4,25



Tourniquet q = 2 — attente : A = 9, B = 2, C = 10, D = 10 → 7,75



04

En pratique sous Linux

CFS, EEVDF, temps réel et valeur nice

L'ordonnanceur de Linux

Classe	Pour	Algorithme
`SCHED_DEADLINE`	tâches à échéance garantie	EDF (échéance la plus proche)
`SCHED_FIFO / SCHED_RR`	temps réel, priorités 1 à 99	FIFO ou tourniquet par priorité
`SCHED_OTHER`	tous les processus ordinaires	CFS (2007), puis EEVDF (noyau 6.6, 2023)
`SCHED_BATCH / SCHED_IDLE`	calculs de fond, tâches inactives	variantes à faible priorité

L'idée de CFS

Chaque processus a un « temps virtuel » d'exécution. On élit celui qui a le moins reçu : chacun obtient une part équitable, pondérée par sa valeur nice. Les processus sont rangés dans un arbre rouge-noir.

EEVDF (depuis 2023)

Garde l'équité de CFS, mais tient aussi compte d'une échéance virtuelle : les tâches sensibles à la latence (interactives, audio) passent plus vite.

Agir sur la priorité : nice et renice

- La valeur nice va de -20 (prioritaire) à 19 (très gentil avec les autres) ; 0 par défaut
- Un utilisateur peut seulement augmenter sa valeur nice ; la baisser est réservé à root
- En C : nice(incrément) ou setpriority()

terminal

```
$ nice -n 10 ./calcul &        # lancer avec nice = 10
[1] 5120
$ renice -n 15 -p 5120          # changer en cours de route
5120 (process ID) old priority 10, new priority 15
$ ps -o pid,ni,cmd -p 5120
  PID  NI CMD
 5120  15 ./calcul
$ sudo chrt -f 50 ./capteur     # temps réel SCHED_FIFO
```



-20 : prioritaire 0 19 : gentil

Sous CFS, chaque cran de nice change d'environ 10 % la part de processeur : un processus à nice 0 face à un processus à nice 5 obtient environ 3 fois plus de temps.

SYNTHÈSE

À retenir

01

Le rôle

Choisir, parmi les prêts, qui obtient le processeur : à la création, fin, blocage, fin d'E/S, horloge.

02

Les critères

Séjour = fin − arrivée ; attente = séjour − exécution. Débit, réponse, équité.

03

FIFO et SJF

Non préemptifs. FIFO : effet convoi. SJF : optimal en attente, mais il faut les durées.

04

Tourniquet

Préemptif, réactif. Le quantum est un compromis entre réactivité et coût des commutations.

05

Priorités

Plusieurs files. Le vieillissement évite la famine.

06

Linux

CFS puis EEVDF pour les processus ordinaires, classes temps réel, nice de -20 à 19.

Ressources pédagogiques

Le manuel et les outils

```
$ man 7 sched
$ man 1 nice
$ man 1 chrt
$ top            # colonnes PR et NI
$ cat /proc/<pid>/sched
```

man 7 sched décrit toutes les classes d'ordonnancement de Linux et leurs règles.

Lectures

- A. Tanenbaum, Systèmes d'exploitation : « Ordonnancement »
- R. et A. Arpaci-Dusseau, Operating Systems: Three Easy Pieces (gratuit en ligne) : chapitres 7 à 9
- Documentation du noyau : CFS Scheduler, EEVDF Scheduler

lamae

Questions ?

PROCHAINE SÉANCE

CM 07a — La mémoire virtuelle (1/2)

va-et-vient · MMU · segmentation · pile et tas

Christophe Cruz · christophe.cruz@ube.fr